Sanhita

Pan-Asia legal research counsel — what we built, what we ship next, what it costs.

Sanhita — Pan-Asia Legal Research Counsel

> ***What it is:*** *an AI legal research assistant for advocates in* ***India, Singapore, and Hong Kong****. Ask a question → get a grounded answer with case citations, draft contracts, search a live court-case database (1,135 cases ingested from open GitHub datasets), redline documents, draft notices, translate into 33 languages, ship the result by email or Google Doc, and run an in-house admin dashboard with realtime updates over Socket.io.*

Table of contents

1. *What we built (capability inventory)*
2. *Architecture — how the pieces fit*
3. *Tech stack*
4. *What it can do today*
5. *Numbers (v2 snapshot)*
6. *What we can build next (12-week roadmap)*
7. *What's needed (infra, accounts, data)*
8. *Cost model*
9. *Local development*
10. *Deployment notes*
11. *Security & data handling*

1. What we built (capability inventory)

1.1 The chat itself (Assistant pane)

- Multi-mode chat — research (BM25 + connectors), draft (open-canvas, no retrieval), web (Tavily/Serper/Gemini-grounded snippets), agent (tool-using loop).
- Live "thinking" panel — ChatGPT-style phase strip ("Searching the case-law index…" → "Drafting the memo…" → "Checking citations resolve…").
- Citation chips — every `[n]` in an answer is a clickable superscript that highlights the matching source card.
- Multi-section memos — system prompt enforces a 7-section structure (Overview, Statutory Framework, Key Concepts, Procedure, Penalties, Recent Developments, Practice Pointers).
- Action row on every answer — Copy (plain text), Email (mail client pre-filled), Save as Doc (Google Docs API or .md fallback).

- Suggested next steps — three Harvey-style follow-up cards in a 2-column grid.
- Model picker — Auto · Gemini 2.5 Flash (default), Claude Sonnet 4.5, Llama 3.3 70B (Groq), Workers AI.
- Language picker — 33 languages including all 22 Eighth Schedule Indian languages, routed through Sarvam AI for Indian languages.

1.2 Court Search pane (NEW in v2)

- Dedicated sidebar pane between Workflows and Clients.
- **Search tab** — full-text BM25 over the live GitHub-ingested corpus, jurisdiction (IN/SG/HK) and tier (SC, HC, CFA, CA, CFI…) filters, 20 results per page.
- **Latest tab** — newest cases ingested per jurisdiction, sorted by `(added_at, year)` desc.
- **Case detail drawer** — title, court · year · citation, full text up to 4000 chars, "Open original" link, "Use in Assistant" handoff.
- **Compare cases** — multi-select up to 4 cases, click "Compare in Assistant" → seeds a new chat with the side-by-side analysis prompt.

1.3 Dashboard pane (NEW in v2)

In-house admin panel mounted in the sidebar.

- **At a glance** — active users, threads, messages, library docs, BM25 corpus size with per-jurisdiction breakdown.
- **System health** — DB mode (sqlite/postgres) + URL host, BM25 status, LLM router providers with keys set, web search reachability.
- **Users widget** — list of access-code holders with last-seen and thread count, one-click revoke.
- **Activity feed** — live audit log streamed via Socket.io. Every connector-key set/delete and user revoke fans out to all connected admins instantly. Presence indicator shows how many admins are watching.
- **Ask Sanhita CTA** — pre-fills the Assistant with a snapshot of the current dashboard state (real numbers, not placeholders).

1.4 Storage / Vault

- Multipart upload, list, delete (`/api/vault/*`).
- Per-document Q&A using BM25 over the upload + Gemini synthesis.

1.5 Workflows

- **Draft** — template + facts JSON → drafted document.
- **Review** — clauses → flag unusual terms.
- **Translate** — EN ↔ 33 languages.
- **Citator** — case title + holdings → cite-checked.
- **Redline** — large-text diff → JSON suggestions {remove, replace, add}.
- 9 generic flows wired into one card grid.

1.6 Clients · History · Library · Settings

- **Clients**: inbox of leads pulled from NyayaSathi WhatsApp/voice; click → opens matter as a fresh chat thread.

- **History**: SQLite-backed thread search + date-range filter chips.
- **Library**: curated statutes + contract templates, filterable by jurisdiction (IN/SG/HK) and kind.
- **Settings**: per-connector API key plug-in surface (DB-backed, env fallback). Every change appends to the dashboard activity log.

1.7 Retrieval engine — `retrieval_pkg/`

- `BM25Index` class on `rank_bm25.BM25Okapi`. Pure Python, ~5ms/query at 100K docs.
- `Document` dataclass — fields: `case_id, title, text, court, year, citation, jurisdiction, tier, url, source, added_at, extra`.
- `bm25.pkl` — pickled corpus + tokens.
- `add()` is idempotent — re-running an ingestor never duplicates.
- `latest()` powers the Court Search Latest tab.

1.8 Ingestion engine — `scripts/ingestors/` + `scripts/ingest_github_data.py`

Source	Country	Status	Output
`hk_cuthchow_csv`	HK	Live — pulls cuthchow/Hong-Kong-Courts CSV	882 docs
`sg_lacuna`	SG	Live — pulls hueyy/lacuna-db (FC, STC, PDPC, LSS-DT)	239 docs
`india_seed_promote`	IN	Live — promotes `seed_corpus.py` landmarks	14 docs
`hk_ylchan_list`	HK	Stub — upstream is scraper-only	—
`sg_codelah`	SG	Stub — upstream has no legal data	—
`india_openjustice`	IN	Stub — upstream is scraper-only	—
`india_vanga_hc`	IN	Stub — upstream is on AWS S3 (1.1TB)	—

Driver supports `--source NAME`, `--all`, `--limit N`, `--top-up`. Atomic save (`.tmp` → rename).

1.9 LLM router — `llm/router.py`

- Provider chain: Gemini 2.5 Flash → Anthropic Claude Sonnet 4.5 → Groq Llama 3.3 70B → Cloudflare Workers AI.
- Circuit-breaker per provider.
- `prefer="X"` reorders the chain.

1.10 Validation gates — `validators/answer_gates.py`

- 6 gates: G1 citation present, G2 citation resolves, G3 banned phrases, G4 grounding ratio ≥ 60%, G5 no fabricated case names, G6 quote spans match.

- Per-mode: research = all 6, draft = G3 only, web = G1+G2+G3.

1.11 Realtime layer (NEW in v2) — `realtime.py`

- python-socketio mounted on FastAPI. Single uvicorn process serves both HTTP and WebSocket.
- Events: `activity:append`, `presence:join`, `presence:leave`, `bm25:reloaded`, `settings:changed`.
- Best-effort `broadcast()` safe to call from threadpool handlers.

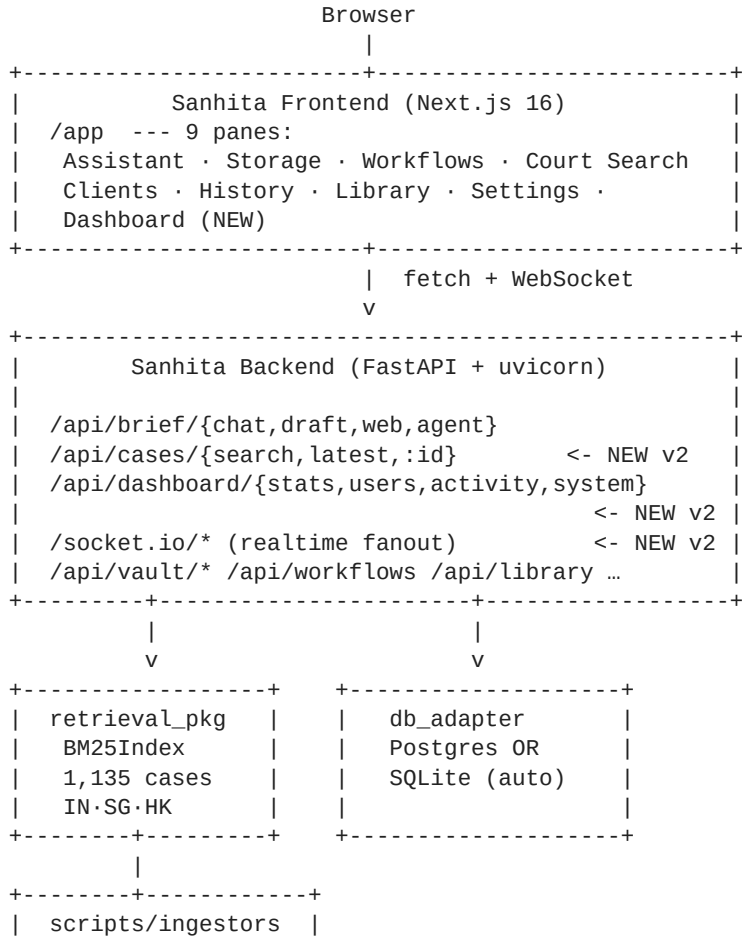
1.12 DB adapter (NEW in v2) — `db_adapter.py`

- Routes to **Postgres (NeonDB)** when `DATABASE_URL` is set, else falls back to SQLite.
- `psycopg[binary]` 3.x with a connection pool.
- Cross-driver `q()` rewrites `?` → `%s` for Postgres.
- Owns the dashboard schema (`dash_activity`, `dash_settings`).

1.13 Other infra

- SQLite (`lexsearch.db`) for auth, sessions, threads, messages, library_docs, connector_keys, clients.
- Three rate-limit buckets (chat, draft, web, agent).
- Input guards strip Aadhaar/PAN/phone before storage.

2. Architecture — how the pieces fit



```
|   hk_cuthchow_csv   |
|   sg_lacuna         |
|   india_seed_*      |
+--------+------------+
         |
         v
 GitHub raw content
 (CSV / JSON dumps)
```

3. Tech stack

Layer	Choice	Why
Frontend	Next.js 16 (App Router, Turbopack)	Fast HMR, native font optimisation
Styling	Tailwind 4 + custom CSS variables	Beige/serif legal aesthetic; Fraunces (display) + Inter (body)
Backend	FastAPI + uvicorn	Async, easy to deploy
Realtime	python-socketio + socket.io-client (NEW v2)	Audit-log fanout, presence
DB	NeonDB Postgres in prod, SQLite in dev (auto)	Zero-config dev; HA Postgres prod
Auth	Access-code + signed cookies	OAuth on the roadmap
LLM	Gemini 2.5 Flash (primary)	Best quality/$ for grounded Q&A
LLM fallback	Anthropic, Groq, Cloudflare	Circuit-breaker degrades gracefully
Translation	Sarvam AI `mayura:v1`	Indian-language quality
Retrieval	`rank_bm25` + pickle	Pure-Python, no infra
Web search	Tavily → Serper → Gemini grounded → DuckDuckGo	Tier-falls-through

4. What it can do today

User says	What happens
"Limitation period for §138 NI Act?"	BM25 → 6 hits → Gemini drafts a 7-section memo with `[n]` citations
"Draft a 2-clause NDA between Acme and Kyoto Holdings"	Skips retrieval, drafts a 4,400-char NDA in ~6s
"Latest 2025 SC ruling on PMLA bail"	Web mode — Tavily / Serper / Gemini grounded → snippet-cited answer
"Find me HK Court of First Instance defamation cases"	Court Search → BM25 over 882-doc HK corpus → ranked cards
"Compare these 3 maintenance cases"	Select 3 in Court Search → Compare → side-by-side analysis
"Use this case in my chat"	Seeds a fresh thread with the case as context
"Translate the answer into Tamil"	Reply language flipped → Sarvam translates the markdown post-hoc
"Save this as a Google Doc"	If Google connected → opens in Docs; else → `.md` download
"Email this to my client"	Opens mail client with answer pre-filled

User says	What happens
"Redline this NDA"	Workflows pane → JSON of {remove, replace, add} suggestions
"Add my Indian Kanoon API key"	Settings → DB keystore + broadcast to other admins live
Admin opens Dashboard	4 widgets render with real data; "1 live" presence chip; Activity feed updates without F5

5. Numbers (v2 snapshot)

Metric	Value
Lines of code (Python + TS)	~28,000
Sidebar panes	9
API endpoints	48+
LLM providers wired	4
Translation languages	33
Court cases indexed today	**1,135** (HK 882 · SG 239 · IN 14)
Sources of case data	3 live ingestors + 4 stubs
Time per answer (Gemini Flash, research)	6-10s
Time per answer (Gemini Flash, draft)	8-15s
Cost per answer	~$0.001
Realtime events broadcasting	activity, presence, settings, bm25:reloaded

6. What we can build next (12-week roadmap)

Phase 1 — finish the corpus (week 1-3)

- Ingest the rest of the 6 GitHub sources to grow corpus from 1,135 → ~110K:
- `india_vanga_hc` — 80K Indian HC judgments via AWS S3 sync
- `india_openjustice` — 15K openjustice-in/* dumps via the `ecourts` library
- `hk_ylchan_list` — wire its scraper output into the ingestor
- Nightly cron — `scripts/ingest_github_data.py --all --top-up`.

Phase 2 — semantic recall (week 3-5)

- Pipe each Document through Gemini's `text-embedding-004` (768-dim).
- Store vectors in Qdrant Cloud.
- Hybrid retrieval — BM25 + cosine similarity → RRF (k=60).

Phase 3 — Call surface (NEW priority)

See `CALL_SURFACE_PLAN.md` for the full design — voice/WhatsApp/Twilio surface that lets advocates query Sanhita by phone.

Phase 4 — productisation (week 5-8)

- Multi-tenant — strip the single-org assumption.
- Postgres migration — `DATABASE_URL` is wired; just point at Neon.
- Stripe billing.
- Org admin pane — invite users, set roles.

Phase 5 — agent muscle (week 8-10)

- Agent clarifies before drafting.
- Multi-document agent for Vault.
- Streaming responses (SSE).

Phase 6 — distribution (week 10-12)

- NyayaSathi → Sanhita redirect.
- Mobile responsive polish.
- Custom domain + outbound email.

7. What's needed (infra, accounts, data)

Required to ship the v1

Item	Source	Cost (per month)
Gemini API key	aistudio.google.com	$0 free tier
Hosting — backend	Railway / Render	$7-25
Hosting — frontend	Vercel hobby	$0
Domain	sanhita.ai	$4
Persistent disk for `bm25.pkl`	Railway/Render volume 5GB	$1.50

For serious traction (week 4+)

Item	Source	Cost (per month)
NeonDB Postgres	neon.tech	$0 → $19
Qdrant Cloud	qdrant.cloud	$0 → $25
Tavily web search	tavily.com	$0 → $30
Serper backup	serper.dev	$0 → $50
Sarvam AI translate	sarvam.ai	usage-priced
Indian Kanoon API key	indiankanoon.org/api	~$24
Stripe	stripe.com	2.9% + 30¢

Item	Source	Cost (per month)
Resend email	resend.com	$0 → $20
Sentry	sentry.io	$0 → $26

For the Call surface (Phase 3)

Item	Source	Cost
Twilio voice + SMS	twilio.com	$0.014/min IN, $0.0075/SMS
WhatsApp Business API	Meta + BSP	$0.0065/conversation
Indian phone number (DID)	Twilio India	~$1/mo
Voice TTS (Indian languages)	Sarvam Bulbul / Google TTS	usage-priced
Voice STT	Sarvam Saaras / Google STT	usage-priced

8. Cost model — pilot vs scale tiers

Pilot (0-50 active lawyers, ≤5K queries/month)

- Hosting (Railway starter + Vercel hobby): **$15**
- Domain: **$4**
- Gemini Flash @ 5K queries: **$3-5**
- Sarvam translate (occasional): **$2**
- **Total ~$25/month**

Scale (500 active lawyers, 50K queries/month)

- Hosting Pro tier: **$70**
- Postgres (Neon scale): **$19**
- Qdrant Cloud: **$25**
- Gemini @ 50K queries: **$30-50**
- Anthropic fallback (~5%): **$15-25**
- Tavily 10K: **$30**
- Serper 50K: **$50**
- Sarvam translate (5K): **$10-15**
- Indian Kanoon API: **$24**
- Resend email: **$20**
- Sentry: **$26**
- Stripe fees (~$1900 rev × 3% + $0.30): **$87**
- **Total ~$420-500/month**

At $19/seat × 500 = $9,500/mo revenue, infra is ≤ 6%.

Per-query cost (Gemini Flash 2.5)

- Input ~2K tok → $0.0003
- Output ~1.2K tok → $0.00072
- **Per answer ~$0.001**. 1,000 answers = $1.

9. Local development

```
git clone https://github.com/Pranav0509-stack/sanhita.git
cd sanhita

python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

cd web && npm install && cd ..

cp .env.example .env
# edit .env — at minimum set GEMINI_API_KEY

python3 scripts/ingest_github_data.py --all

# Backend (port 8080) — note the socketio_app target for realtime
LEXSEARCH_BM25_ENABLED=true uvicorn server:socketio_app --port 8080

# Frontend (port 3001) — separate terminal
cd web && BACKEND_ORIGIN=http://localhost:8080 npm run dev -- --port 3001

# Open http://localhost:3001/login → SNHT-DEMO-2026
```

10. Deployment notes

Recommended target: Railway

- Backend service: Dockerfile build, healthcheck `/health`, persistent disk at `/data` for `bm25.pkl`.
- Frontend service: NIXPACKS auto-build, `npm run start -- --port $PORT`.
- `railway.toml` (root) and `web/railway.toml` already in repo.
- Set `DATABASE_URL` to your Neon connection string to flip from SQLite → Postgres.

Alternatives

- Render — `render.yaml` + `Procfile` already in repo.
- Fly.io — for per-region BM25 sharding.

11. Security & data handling

- No live keys in the repo. `.claude/launch.json` is gitignored. Production keys in encrypted env-vars.
- Per-user threads — every `/api/brief/threads/:id` query checks `messages.user_id`.

- PII redaction — `validators/input_guards.py` strips Aadhaar / PAN / phone numbers.
- Google Workspace OAuth uses standard offline flow; tokens encrypted in `auth.connector_keys`.
- Rate limits — chat 30/min, draft 20/min, web 15/min, agent 10/min per IP.
- Validation gates — answers fail closed (refusal) if grounding ratio < 60% in research mode.
- Dashboard audit log — every admin write appended to `dash_activity` and broadcast over Socket.io.

Last updated: 2026-04-27 — v2.

Sanhita — Call surface plan

> ***Goal:*** *let an Indian advocate query Sanhita by* ***phone call or WhatsApp voice note****, in their own language, and get back a grounded legal memo by SMS / WhatsApp / call-back. Same answers, same citations, same Gemini brain — just a non-browser entry point.*

This doc is self-contained for a fresh Claude session. Read it, then read `SANHITA.md` for the existing app surface.

What "the call part" means

Three distinct surfaces to build, in priority order:

1. **Inbound voice call** — advocate dials a Twilio Indian DID, speaks

their question (Hindi / Tamil / English / etc.), gets the answer read back by TTS in the same language, with the full memo + cites sent via SMS / WhatsApp afterwards.

2. **Inbound WhatsApp voice note** — same model but async: lawyer

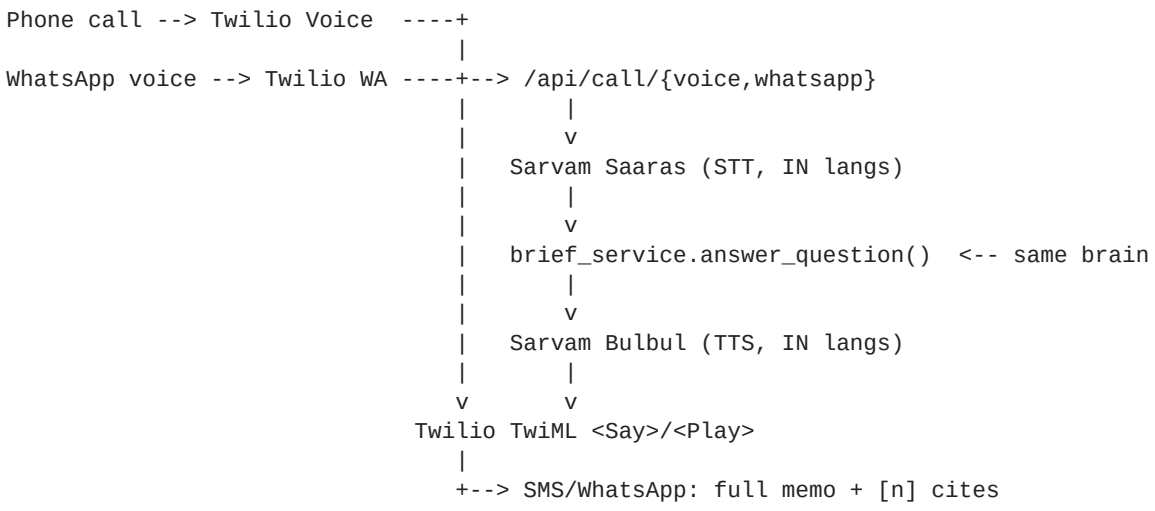
sends a WhatsApp voice note, Sanhita transcribes (Sarvam Saaras), answers, replies with the audio answer + a text memo.

3. **Outbound call-back from the web app** — from the Assistant pane,

click "Call me with the answer" → Twilio places an outbound call reading the latest answer aloud in the chosen language. Useful for advocates driving to court who already have a thread open.

All three reuse the **existing** `brief_service.answer_question` / `answer_open` **pipeline** — no new LLM, no new validator. Just adapters.

Architecture



What to build

Phase A — Twilio inbound voice (~3 days)

New file: `call_service.py`

- `handle_incoming_call(from_number, language_hint) -> TwiML`
- Greet caller in Hindi by default ("Sanhita mein swagat hai. Apna sawal poochiye.")
- `<Gather input="speech" speechTimeout="auto" language="hi-IN" action="/api/call/voice/answer">`
- `handle_voice_answer(speech_result, from_number) -> TwiML`
- Light-weight intent detection (chit-chat vs research) using the existing `_is_chitchat` / `_detect_intent` from `server.py`.
- Call `brief_service.answer_question(speech_result, hits=[], language=detected_lang) – hits=[]` lets the BM25 path fire if enabled, otherwise the model just talks.
- Truncate the answer to the first 2 sentences for `<Say>`. Send the full memo over SMS / WhatsApp follow-up.
- Persist as a row in a new `call_transcripts` SQLite table so the Dashboard's Activity feed shows "+91-9876543210 asked about anticipatory bail".

New endpoints (`server.py`):

- `POST /api/call/voice` — Twilio Voice webhook
- `POST /api/call/voice/answer` — speech-result handler
- `POST /api/call/voice/sms-followup` — sends the full memo as SMS

TwiML helpers — `twilio` SDK is already on PyPI. Add `twilio>=9.3.0` to `requirements.txt`.

Twilio config:

- Buy an Indian DID (~$1/mo) at console.twilio.com.
- Set the Voice URL to `https://api.sanhita.ai/api/call/voice`.
- Enable speech recognition for Hindi, Tamil, Telugu, Bengali, English (multi-lang Gather is supported in Twilio).

Smoke test:

- Buy a $1 Indian Twilio DID.
- Call it from your phone.
- Speak: "Section 138 ki limitation kya hai?"
- Expect: Hindi voice answer "138 ke liye limitation 30 din hain notice ke baad…" + SMS with full memo + cites.

Phase B — WhatsApp voice note (~2 days)

Twilio WhatsApp Sandbox is free; production needs a BSP.

- New endpoint: `POST /api/call/whatsapp` — Twilio WA webhook.
- If message contains a `MediaUrl0`, fetch the audio, run through

Sarvam Saaras STT, then `answer_question`, then send back two WhatsApp messages: (1) Sarvam Bulbul TTS audio file, (2) text memo with cites.

- If text-only message, skip STT, go straight to answer.

Sarvam wiring:

- `llm/sarvam.py` already has `translate()`. Add `stt(audio_bytes,

language) -> str and tts(text, language) -> bytes helpers using the same mayura:v1` API surface.

Phase C — Outbound call-back from web (~1 day)

Frontend:

- In `ChatBubble` action row (next to Copy / Email / Save as Doc),

add a "Call me" button that opens a small modal asking for a phone number + language.

Backend:

- `POST /api/call/outbound` — `{phone, message_id, language}` →

`twilio.client.calls.create(to=phone, from_=DID, url=…)` where the URL hosts a TwiML that `<Say>`s the answer in the chosen language.

Cost guard: rate-limit to 3 outbound calls per user per hour.

Files to create / modify

File	What
NEW `call_service.py`	TwiML builders + STT/TTS bridges
NEW `llm/sarvam.py` (extend)	Add `stt()` and `tts()` helpers
`server.py`	Add `/api/call/{voice,voice/answer,whatsapp,outbound}` endpoints
`auth.py`	Add `call_transcripts` table
`requirements.txt`	Add `twilio>=9.3.0`
`.env.example`	Add `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_VOICE_DID`, `TWILIO_WA_FROM`
`web/src/app/app/page.tsx`	"Call me" button in `ChatBubble` action row

Existing code to reuse

- `brief_service.answer_question()` — drop in directly.
- `_is_chitchat()` and `_detect_intent()` from `server.py`.
- `validators/input_guards.py` — same length / scope / PII guards.
- `llm/sarvam.py` `translate()` — pattern to follow for `stt()`/`tts()`.
- `realtime.broadcast()` — fan out a `call:answered` event so the

Dashboard shows live call activity alongside web admin actions.

Costs

Item	Cost
Twilio India voice (inbound)	$0.014/min
Twilio India SMS	$0.0075/SMS
Twilio India DID	~$1/mo
WhatsApp Business API	$0.0065/conversation
Sarvam Saaras (STT)	usage-priced (~$0.001/sec)
Sarvam Bulbul (TTS)	usage-priced
Gemini Flash answer	~$0.001/answer

Per call (90 sec, ~30 sec speech, 60 sec TTS playback, 1 SMS):

- Voice: $0.014 × 1.5 = $0.021
- STT: $0.001 × 30 = $0.030
- TTS: ~$0.005
- Gemini: $0.001
- SMS: $0.0075
- **~$0.065 per call**

At ■5 (~$0.06) revenue per call (typical Indian micro-payment), the unit economics work if 90%+ of calls convert to a satisfactory answer.

Smoke checklist

- [ ] Twilio Indian DID purchased and pointed at `/api/call/voice`.
- [ ] `TWILIO_*` env vars set in Railway / Render.
- [ ] `requirements.txt` has `twilio>=9.3.0`; `pip install` clean.
- [ ] `server.py` route `POST /api/call/voice` returns valid TwiML

(use `curl -X POST` with form data shaped like Twilio's webhook).

- [ ] End-to-end: call the DID, ask a §138 NI Act question, hear an

answer in Hindi, receive an SMS with the full memo.

- [ ] Dashboard's Activity feed shows the call event live (Socket.io

`call:answered` broadcast).

Out of scope for this work-stream

- Native iOS/Android app (Phase 7+).
- IVR menus / multi-step voice flows (just a single Q→A loop).
- Voice-to-voice conversation memory across calls (each call is its

own thread; tie to a user via `from_number` lookup).

- Compliance: TRAI DLT for SMS in India is required for outbound SMS

to non-Twilio-customers. Inbound is fine; outbound needs a header registered with one of the operator panels (Airtel/Jio/Vi).